

Design Document: CampusGo - Real-Time Campus Mobility Platform

Project: Real-Time Campus Mobility and Ride Management Platform

Repository: github.com/code84/CampusGo

Team Members: Prachi Gupta 23118062, Pranshu Sharma 23118063, Raj Rabidas 23118064

Date: June 12, 2026

1. Problem Understanding

1.1 Core Challenge

Large campuses like IIT Roorkee face significant last-mile connectivity issues. Existing e-rickshaw systems operate on fragmented, manual processes leading to:

- **Inefficient Resource Utilization:** Drivers idle in low-demand areas while passengers wait elsewhere.
- **Poor User Experience:** No visibility of driver locations or estimated arrival times.
- **Lack of Accountability:** No ride history, ratings, or feedback mechanism.

1.2 Solution Scope

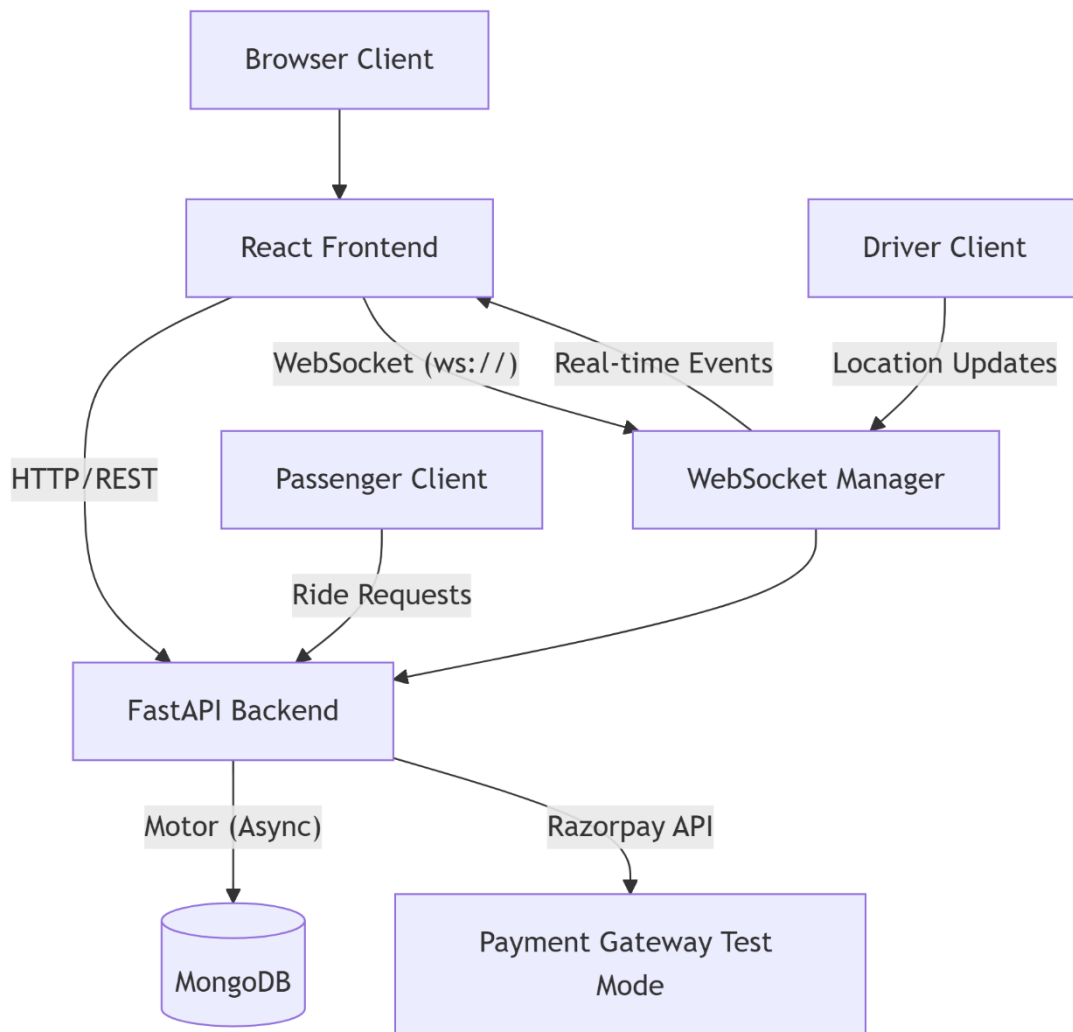
CampusGo addresses these gaps by providing a centralized digital platform with four key pillars:

1. **Real-Time Coordination:** WebSocket-based live updates between passengers, drivers, and the system.
2. **Structured Workflow:** A finite state machine to manage ride lifecycle from requested to completed/cancelled.
3. **Role-Based Access:** Distinct, secure portals for Passengers, Drivers, and Admins.
4. **Actionable Analytics:** Dashboards providing insights into ride patterns and driver performance.

2. System Architecture

CampusGo employs a **modern full-stack, event-driven architecture** to ensure real-time performance and maintainability.

2.1 High-Level Architecture Diagram (Conceptual)



2.2 Technology Stack Justification

Layer	Technology	Justification
Backend	Python FastAPI	High performance for I/O-bound operations (WebSockets, DB), async support, automatic OpenAPI docs.
Database	MongoDB	Flexible schema for varied ride states, geospatial index support for location queries, horizontal scalability.
Real-time	FastAPI WebSockets	Native WebSocket support with simple integration into the same API server (no extra infra like Socket.IO).
Frontend	React 18 + Tailwind CSS	Component-based UI for rapid development, efficient state management via Context API, utility-first CSS for consistent design.
Maps	React-Leaflet + OSRM	Open-source, cost-effective, works with dark mode design system.

Auth	JWT + bcrypt	Stateless authentication suitable for horizontal scaling, secure password hashing.
------	--------------	--

2.3 Data Flow: Ride Request Example

1. Passenger requests ride → POST /api/rides/request.
2. Backend creates ride with status requested, stores in MongoDB.
3. System publishes new_ride_request event via WebSocket to all online, verified drivers.
4. Driver accepts → POST /api/rides/{id}/accept with atomic update to prevent double assignment.
5. Passenger receives ride_accepted WebSocket event → UI updates.
6. Passenger pays → POST /api/rides/{id}/payment → status paid.
7. Driver starts ride → status in_progress → completes → status completed.
8. Passenger rates → feedback stored, driver rating updated.

3. Database Schema

We use **MongoDB** with 5 main collections. Schemas are enforced using Pydantic models in the backend.

3.1 users Collection

Stores all authenticated entities (passenger, driver, admin) with role-based fields.

```
{
  "_id": ObjectId,
  "email": "string (unique)",
  "password_hash": "string",
  "role": "passenger | driver | admin",
  "full_name": "string",
  "created_at": "ISO Date",
  // Driver-specific fields
  "vehicle_number": "string?",
  "is_verified": "boolean (default false)",
  "verification_status": "pending | approved | rejected",
  "avg_rating": "float",
  "total_rides": "int",
  // Location & status
  "current_location": { "type": "Point", "coordinates": [lng, lat] },
```

```
"is_online": "boolean"
```

```
}
```

3.2 rides Collection

Core entity capturing the complete ride lifecycle.

json

```
{
```

```
"_id": ObjectId,
```

```
"passenger_id": ObjectId(ref: users),
```

```
"driver_id": ObjectId(ref: users, nullable),
```

```
"pickup_location": { "name": "string", "coordinates": [lng, lat] },
```

```
"destination": { "name": "string", "coordinates": [lng, lat] },
```

```
"status": "requested | accepted | in_progress | completed | cancelled",
```

```
"scheduled_for": "ISO Date? (for future rides)",
```

```
"requested_at": "ISO Date",
```

```
"accepted_at": "ISO Date?",
```

```
"completed_at": "ISO Date?",
```

```
"fare_estimate": "int (₹)",
```

```
"payment_status": "pending | paid",
```

```
"payment_id": "string? (Razorpay)",
```

```
"rating": "int (1-5)?",
```

```
"feedback": "string?"
```

```
}
```

3.3 notifications Collection

Powers the persistent inbox and unread counts.

json

```
{
```

```
"_id": ObjectId,
```

```
"user_id": ObjectId(ref: users),
```

```
"type": "ride_update | payment | verification",
```

```
"title": "string",
```

```
"message": "string",
```

```
"is_read": "boolean",
```

```
"created_at": "ISO Date",
```

```
"metadata": { "ride_id": ObjectId?, "driver_id": ObjectId? }
}
```

3.4 verification_requests Collection

Manages driver onboarding queue separately from user document.

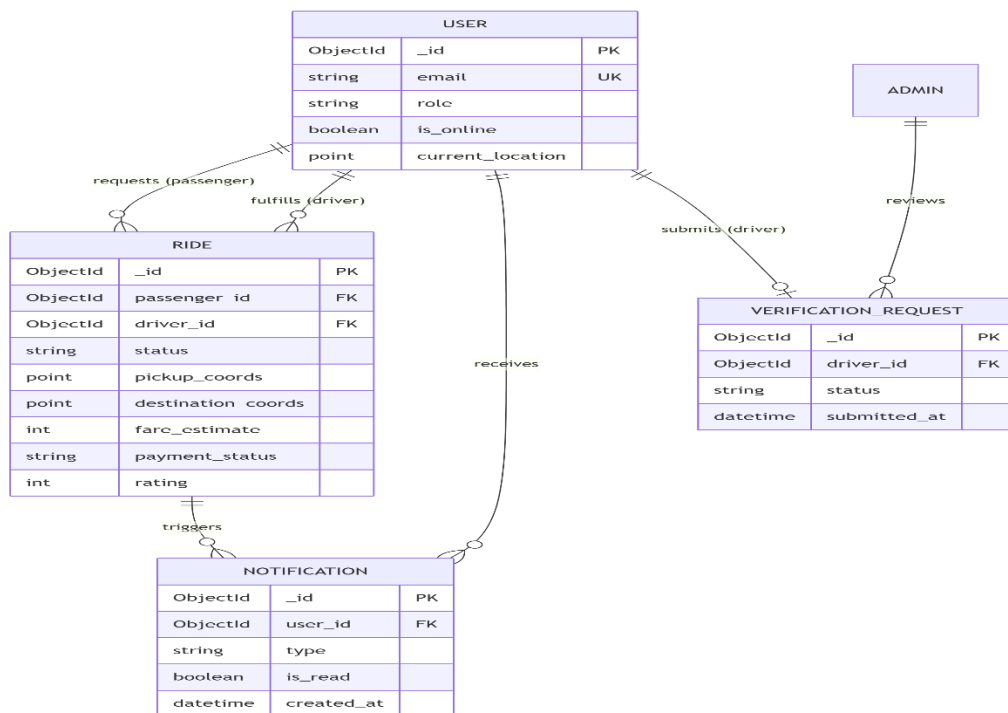
json

```
{
  "_id": ObjectId,
  "driver_id": ObjectId(ref: users),
  "submitted_at": "ISO Date",
  "admin_notes": "string?",
  "status": "pending | approved | rejected"
}
```

3.5 analytics_cache Collection (Optional)

Pre-aggregated data for demand charts (peak hours, popular locations) to avoid real-time map-reduce.

4. Entity Relationship Diagram (ERD)



Key Relationships:

- **One-to-Many:** A user (passenger) can have many rides. A driver can have many rides.
- **One-to-One (logical):** A ride can have one payment status. A verification request is linked to one driver.
- **Referential Integrity:** Maintained at the application level using MongoDB DBRefs or manual reference validation.

5. API Overview

The REST API is organized by resource and follows standard conventions. **All endpoints are mounted under /api** and return JSON.

5.1 Authentication & Users

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register passenger or driver.
POST	/auth/login	None	Returns JWT access token.
GET	/auth/me	JWT	Get current user profile.

5.2 Ride Management (Core Workflow)

Method	Endpoint	Auth	Description
POST	/rides/request	Passenger	Create new ride request.
GET	/rides/available	Driver	Fetch unassigned ride requests.
POST	/rides/{id}/accept	Driver	Atomically accept a ride (prevents conflicts).
POST	/rides/{id}/start	Driver	Change status → in_progress (requires payment).
POST	/rides/{id}/complete	Driver	Change status → completed.
POST	/rides/{id}/cancel	Passenger/Driver	Cancel before completion.
POST	/rides/{id}/rate	Passenger	Submit rating & feedback.

5.3 Real-time WebSocket

Event Type (Client → Server)	Description
driver_location_update	Driver sends current GPS coordinates.
driver_toggle_online	Driver goes online/offline.
Event Type (Server → Client)	Description
new_ride_request	Broadcast to online drivers.
ride_accepted	Sent to passenger and other drivers.
ride_status_update	Status change for involved parties.
payment_confirmation	Notify driver after payment.

5.4 Admin & Analytics

Method	Endpoint	Auth	Description
GET	/admin/stats	Admin	Platform-wide KPIs.
GET	/admin/pending-drivers	Admin	Queue of unverified drivers.
POST	/admin/drivers/{id}/approve	Admin	Grant driver verification.
GET	/analytics/driver	Driver	Personal performance metrics.
GET	/analytics/passenger	Passenger	Ride history & patterns.

6. Design Decisions

6.1 Critical Decision: Atomic Ride Acceptance

Problem: Two drivers accepting the same ride simultaneously.

Solution: Used MongoDB's `findOneAndUpdate` with a query filter `{ _id: ride_id, status: "requested" }` and update `{ status: "accepted", driver_id: driver_id }`. This atomic operation ensures only one driver succeeds.

6.2 WebSocket vs. Polling

Decision: Chose **WebSockets** for all real-time features.

Rationale:

- **Efficiency:** Reduces server load compared to HTTP polling.
- **Latency:** Instant push notifications for ride requests and status changes.
- **Bi-directional:** Enables live driver location streaming to passengers.

6.3 Driver Verification Queue

Decision: New drivers cannot accept rides until approved by admin.

Justification: Mandatory requirement for campus safety and accountability. Implemented as a `verification_status` field on the user document and middleware `require_verified_driver` for protected routes.

6.4 Payment-Before-Start Workflow

Decision: Ride cannot transition to `in_progress` until `payment_status = paid`.

Rationale: Mirrors real-world commercial ride-hailing trust model. Protects drivers from unpaid trips. Integrated Razorpay test checkout for demonstration, but enforced server-side.

6.5 Frontend State Management

Decision: Used React Context API (`AuthContext`, `SocketContext`) instead of Redux.

Rationale: The application scale (< 15 pages, limited global state) did not justify Redux boilerplate. Context provided sufficient state management for auth token, WebSocket instance, and user role.

6.6 Geospatial Data Handling

Decision: Stored pickup/destination as GeoJSON Point objects.

Rationale: MongoDB supports geospatial indexes (2dsphere), enabling future features like "find nearest driver" or "popular pickup zones" using \$near queries.

6.7 Testing Strategy

Decision: Separate test suites for backend (pytest) and frontend (React Testing Library).

Coverage: API endpoint testing (auth, ride workflow), WebSocket connection simulation, and critical UI component rendering.

7. Security & Scalability Considerations

7.1 Security Measures

- **Password Hashing:** bcrypt with salt factor 12.
- **JWT Storage:** httpOnly cookie recommended (backend fallback) or Authorization: Bearer header.
- **Input Validation:** All requests validated using Pydantic models.
- **CORS:** Restricted to configured origins (CORS_ORIGINS env var).
- **No Secret Exposure:** .env files excluded via .gitignore; Razorpay key secret is server-side only.

7.2 Scalability Approach

- **Stateless Backend:** Any number of FastAPI instances behind a load balancer.
- **WebSocket Sticky Sessions:** Required if scaling horizontally; current single-instance is sufficient for campus-scale.
- **Database Indexes:** Created on rides.status, users.is_online, users.role for query performance.
- **MongoDB Connection Pooling:** Handled by Motor client.

7.3 Future Enhancements (Bonus Features)

1. **Live Map Integration** (Already implemented with Leaflet).
2. **Ride Scheduling** (Schema includes scheduled_for field; backend logic partially ready).
3. **Demand Analytics** (analytics_cache collection ready for hourly aggregation).
4. **Machine Learning Forecast:** Can be added as a separate microservice reading from analytics_cache.

